SENDBIRD AI AGENT · MESSENGER SDK

대화 목록 헤더 버튼 커스터마이징 가이드

목록 화면의 **닫기(X)** → **대화 화면으로 돌아가기**

채팅 화면의 **목록 이동(≡)** 아이콘 변경

대상 기능 · 헤더 슬롯 교체 (ConversationHeaderLayout / ConversationListHeaderLayout)

기준 버전 · @sendbird/ai-agent-messenger-react v1.42.0

문서 작성 · 2026-11-06

목차

1. 요청 내용과 결론
2. SDK 헤더 구조
   - 2.1 두 개의 화면과 두 개의 헤더
   - 2.2 헤더 슬롯 교체 방식
   - 2.3 교체 가능한 슬롯 목록
3. 목록 헤더 — 닫기(X)를 뒤로가기로
   - 3.1 기본 동작
   - 3.2 돌아갈 대화를 정하는 API
   - 3.3 구현 코드
4. 채팅 헤더 — 목록 이동(≡) 아이콘 변경
   - 4.1 기본 동작
   - 4.2 구현 코드
5. 메신저에 연결
6. 동작 확인 결과
7. 구현 시 유의 사항

1 요청 내용과 결론

> **요청** — ① 대화 목록 화면 상단 헤더의 `X` 버튼을 누르면 위젯이 닫히는데, 닫는 대신 **직전 채팅 화면으로 돌아가게** 하고 싶다. ② 채팅 화면에서 **목록으로 들어가는 ≡ 아이콘**도 다른 모양으로 바꾸고 싶다.

결론: 두 건 모두 커스텀 작업이 필요하며, 두 건 모두 구현 가능합니다. 동작이나 아이콘을 바꾸는 설정값은 SDK에 없습니다. 다만 SDK가 헤더를 **슬롯(slot) 구조**로 공개하고 있어서, 해당 자리의 컴포넌트를 통째로 교체하는 방식으로 구현합니다. SDK를 수정하거나 내부 구현에 의존할 필요는 없고, **웹(React) 코드만** 수정하면 됩니다.

설정으로 가능한가	아니오 — 관련 설정값이 없습니다.
필요한 작업	웹(React) 컴포넌트 2개 추가 + 메신저에 슬롯 연결 2줄
네이티브 수정	불필요 (Android / iOS 모두 변경 없음)
사용하는 API	모두 SDK가 공개(export)하는 공식 API

2 SDK 헤더 구조

2.1 두 개의 화면과 두 개의 헤더

메신저 위젯 안에는 화면이 두 개 있고, 헤더도 각각 따로 있습니다. 두 헤더는 서로 다른 레이아웃 컴포넌트를 쓰기 때문에 **한쪽만 골라서 바꿀 수 있습니다.**

화면	헤더 컴포넌트	이번에 바꿀 버튼
대화(채팅) 화면	`ConversationHeaderLayout`	왼쪽 ≡ **메뉴 버튼** → 아이콘 변경
대화 목록 화면	`ConversationListHeaderLayout`	오른쪽 **X 닫기 버튼** → 뒤로가기로 교체

2.2 헤더 슬롯 교체 방식

SDK는 헤더의 각 자리를 이름 붙은 슬롯으로 공개하고 있습니다. 바꾸고 싶은 슬롯에 `component` 로 내 컴포넌트를 넘기면 그 자리만 교체됩니다.

```
// 목록 화면 헤더의 오른쪽 영역 교체
<ConversationListHeaderLayout.EndArea component={MyBackButton} />

// 채팅 화면 헤더의 메뉴 버튼 교체
<ConversationHeaderLayout.MenuButton component={MyMenuButton} />
```

교체했더라도 **원래의 기본 컴포넌트가 필요할 때는 다시 꺼내 쓸 수 있습니다.** 아래 경로로 SDK 기본 구현에 그대로 접근합니다.

```
ConversationListHeaderLayout.defaults.components.CloseButton
ConversationHeaderLayout.defaults.components.MenuButton
```

2.3 교체 가능한 슬롯 목록

참고로 `ConversationHeaderLayout` (채팅 화면 헤더)이 공개하는 슬롯은 다음과 같습니다.

슬롯	자리
`StartArea`	헤더 왼쪽 영역 전체
`MenuButton`	대화 목록으로 가는 ≡ 버튼
`TitleArea` / `Title`	가운데 제목 영역
`EndArea`	헤더 오른쪽 영역 전체
`HandoffButton`	상담사 연결 버튼
`ConversationCloseButton`	대화 종료 버튼
`ExpandButton`	메신저 확장 버튼
`CloseButton`	위젯 닫기 버튼
`MemoryIndicator`	메모리 표시

영역 전체 vs 버튼 하나

`EndArea` 처럼 영역을 통째로 교체하면 그 안의 기본 버튼들이 모두 사라집니다. 버튼 하나만 바꾸고 싶다면 `MenuButton` 같은 **개별 슬롯**을 쓰는 편이 안전합니다.

3 목록 헤더 — 닫기(X)를 뒤로가기로

3.1 기본 동작

SDK의 목록 헤더 닫기 버튼은 **위젯 열림 상태를 false 로 만드는** 동작만 합니다. 화면 이동(navigation)과는 무관하므로, 눌렀을 때 위젯 자체가 사라집니다.

```
// SDK 내부 — ConversationListHeaderLayout 의 기본 CloseButton
const CloseButton = () => {
  const { state } = useMessengerContext();
```

```
  return <IconButton type="close" onClick={() => state.setOpened(false)} />;
};
```

> **참고**
>
> 목록 화면으로 들어가는 경로는 헤더 왼쪽의 **메뉴(≡)** 버튼입니다. 즉 사용자는 채팅 → 메뉴 → 목록 순서로 이동했는데, 목록에서 나가는 버튼이 뒤로가 아니라 닫기로 되어 있어 흐름이 끊긴다는 것이 이번 요청의 배경입니다.

3.2 돌아갈 대화를 정하는 API

API	용도
`useConversationListContext()` → `onOpenConversationView(url, status)`	지정한 대화의 채팅 화면으로 이동
`useConversationListContext()` → `listSource.channels`	현재 목록에 로드된 대화 배열
`useMessengerSessionContext()` → `activeChannel`	직전까지 보고 있던 대화

돌아갈 대화는 ① **직전에 보던 대화(** `activeChannel` **)** → ② **목록의 첫 번째 대화** 순으로 고르면 자연스럽습니다. 둘 다 없는 경우(대화 이력이 하나도 없는 신규 사용자)에는 돌아갈 곳이 없으므로 **SDK 기본 닫기 버튼을 그대로 보여주는** 편이 안전합니다. 화살표만 띄워 두면 눌러도 아무 일이 일어나지 않아 오히려 혼란스럽기 때문입니다.

3.3 구현 코드

`web/src/components/ConversationListBackButton.tsx`

```
import {
  ConversationListHeaderLayout,
  useConversationListContext,
  useMessengerSessionContext,
} from '@sendbird/ai-agent-messenger-react';

export function ConversationListBackButton() {
  const { onOpenConversationView, listSource } = useConversationListContext();
  const { activeChannel } = useMessengerSessionContext();

  // 돌아갈 대화 결정 — 직전에 보던 대화 우선, 없으면 목록의 첫 번째
  const first = listSource.channels[0];
  const target = activeChannel
    ? { url: activeChannel.url, status: activeChannel.status }
    : first
      ? { url: first.url,
          status: (first.conversation?.status === 'closed' ? 'closed' : 'open') as 'open' |
'closed' }
      : null;
```

```
  // 돌아갈 대화가 없으면 SDK 기본 동작(위젯 닫기)을 그대로 둡니다.
  if (!target) {
    return <ConversationListHeaderLayout.defaults.components.CloseButton />;
  }

  return (
    <button
      type="button"
      aria-label="대화 화면으로 돌아가기"
      onClick={() => onOpenConversationView(target.url, target.status)}
      style={{ display: 'flex', alignItems: 'center', justifyContent: 'center',
               width: 32, height: 32, padding: 0, border: 'none',
               background: 'transparent', cursor: 'pointer', color: 'inherit' }}
    >
      {/* 닫기(X) 대신 '뒤로' 의미가 드러나도록 화살표를 씁니다. */}
      <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
           stroke="currentColor" strokeWidth="2"
           strokeLinecap="round" strokeLinejoin="round">
        <polyline points="15 18 9 12 15 6" />
      </svg>
    </button>
  );
}
```

4 채팅 헤더 — 목록 이동(≡) 아이콘 변경

4.1 기본 동작

채팅 화면 헤더 왼쪽의 ≡ 버튼은 `useConversationContext()` 가 제공하는 `onNavigateToConversationList()` 를 호출해 목록 화면으로 이동합니다. 이 함수가 없는 구성(목록 기능을 쓰지 않는 경우)에서는 **버튼 자체를 렌더링하지 않습니다.**

API	용도
`useConversationContext()` → `onNavigateToConversationList()`	대화 목록 화면으로 이동

> **없을 수 있는 함수입니다**
>
> `onNavigateToConversationList` 는 **선택적(optional)** 값입니다. 커스텀 버튼에서도 `if (!onNavigateToConversationList) return null;` 처리를 반드시 넣어 SDK 기본 동작과 동일하게 맞춰 주세요. 빠뜨리면 목록 기능이 꺼진 구성에서 **눌러도 아무 일이 없는 버튼**이 노출됩니다.

4.2 구현 코드

`web/src/components/ConversationListMenuButton.tsx`

```tsx
import { useConversationContext } from '@sendbird/ai-agent-messenger-react';

export function ConversationListMenuButton() {
  const { onNavigateToConversationList } = useConversationContext();

  // 목록 기능이 꺼진 구성에서는 SDK 기본 동작과 동일하게 아무것도 그리지 않습니다.
  if (!onNavigateToConversationList) return null;

  return (
    <button
      type="button"
      aria-label="대화 목록 열기"
      onClick={() => onNavigateToConversationList()}
      style={{ display: 'flex', alignItems: 'center', justifyContent: 'center',
               width: 32, height: 32, padding: 0, border: 'none',
               background: 'transparent', cursor: 'pointer', color: 'inherit' }}
    >
      {/* 햄버거(≡) 대신 '목록' 의미가 드러나는 아이콘을 씁니다. */}
      <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
           stroke="currentColor" strokeWidth="2"
           strokeLinecap="round" strokeLinejoin="round">
        <line x1="9" y1="6" x2="20" y2="6" />
        <line x1="9" y1="12" x2="20" y2="12" />
        <line x1="9" y1="18" x2="20" y2="18" />
        <circle cx="4.5" cy="6" r="1.3" fill="currentColor" stroke="none" />
        <circle cx="4.5" cy="12" r="1.3" fill="currentColor" stroke="none" />
        <circle cx="4.5" cy="18" r="1.3" fill="currentColor" stroke="none" />
      </svg>
    </button>
  );
}
```

이미지 아이콘을 쓰고 싶다면

`<svg>` 자리에 `<img src="/icons/list.svg" width={22} height={22} alt="" />` 처럼 이미지를 넣어도 됩니다. 다만 이미지로 바꾸면 `color: 'inherit'` 에 의한 **테마 색 자동 추종이 동작하지 않으므로**, 다크 테마를 함께 쓴다면 테마별 이미지를 따로 준비해야 합니다.

5 메신저에 연결

`web/src/pages/SimpleChatPage.tsx`

```tsx
<FixedMessenger ...>
  {/* 채팅 화면 헤더 */}
  <FixedMessenger.ConversationChildren>
    <ConversationHeaderLayout.MenuButton component={ConversationListMenuButton} />
  </FixedMessenger.ConversationChildren>

  {/* 대화 목록 화면 헤더 */}
```

```
  <FixedMessenger.ConversationListChildren>
    <ConversationListHeaderLayout.EndArea component={ConversationListBackButton} />
  </FixedMessenger.ConversationListChildren>
</FixedMessenger>
```

슬롯을 넣는 위치에 주의하세요

두 블록을 바꿔 넣으면 적용되지 않습니다.

· 채팅 화면 헤더 → `FixedMessenger.ConversationChildren`

· 목록 화면 헤더 → `FixedMessenger.ConversationListChildren`

6 동작 확인 결과

샘플 앱에 위 코드를 적용하고 실제로 확인한 결과입니다.

확인 항목	결과
채팅 화면 헤더 왼쪽	≡ 대신 목록 아이콘 표시
목록 아이콘 클릭 시	대화 목록 화면으로 정상 이동
목록 화면 헤더 오른쪽	`X` 대신 `<` (뒤로) 아이콘 표시
`<` 클릭 시	위젯이 닫히지 않고 **직전 채팅 화면으로 복귀**
복귀 후 상태	기존 대화 내용·입력창 그대로 유지
채팅 헤더의 나머지 버튼	변화 없음 — 기존 버튼 그대로 유지

확인 완료

채팅 ↔ 목록 양방향 이동이 의도대로 작동하며, 이동 중 위젯 열림 상태가 유지되는 것까지 확인했습니다.

7 구현 시 유의 사항

대화 이력이 없는 경우

신규 사용자처럼 대화가 하나도 없으면 돌아갈 대상이 없습니다. 위 코드는 이때 **SDK 기본 닫기 버튼으로 자동 전환**되므로 별도 처리가 필요 없습니다.

목록 필터 설정의 영향

진행 중인 대화만 보이도록 목록 필터(`conversationStatusFilter`)를 걸어 둔 경우, 종료된 대화는 `listSource.channels` 에 포함되지 않습니다. 위 코드는 `activeChannel` 을 먼저 보기 때문에 **종료된 대화를 보던 중이라도 그 대화로 정상 복귀합니다.**

아이콘 디자인

예시는 외부 의존성 없이 인라인 SVG로 아이콘을 그렸습니다. 서비스에서 쓰는 아이콘 세트가 있다면 `<svg>` 부분만 교체하면 됩니다. 버튼 색상은 `color: 'inherit'` 로 두어 헤더 테마 색을 자동으로 따라갑니다.

터치 영역

모바일 웹뷰에서는 버튼 크기를 **최소 32×32px 이상**으로 유지해 주세요. 아이콘 자체를 작게 그리더라도 `<button>` 의 크기는 줄이지 않는 편이 오터치를 줄이는 데 유리합니다.

접근성

`aria-label` 을 반드시 남겨 주세요. 아이콘만 있는 버튼은 스크린 리더가 읽을 텍스트가 없어 레이블이 빠지면 시각 보조 기술 사용자가 용도를 알 수 없습니다.

본 문서는 @sendbird/ai-agent-messenger-react v1.42.0 기준으로 작성되었습니다. 사용된 API는 모두 SDK가 공개적으로 export 하는 항목이며, 내부 구현에 의존하지 않습니다. SDK 버전 업그레이드 시 동작을 재확인하시기 바랍니다.